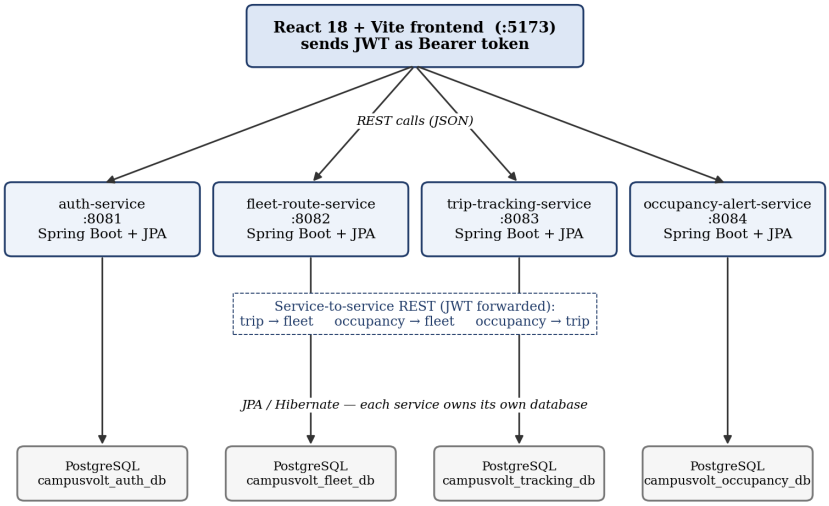
WEB PROGRAMMING

Project Report

CampusVolt

Smart Campus EV Fleet Intelligence System

Four Spring Boot microservices and a React dashboard for running a campus electric shuttle fleet — vehicles and routes, simulated live tracking, passenger occupancy and automatic alerts.



Submitted by

2024115018 — **Kawin A N**

2024115020 — **Shankararam S U**

2024115124 — **Rohan Beniel E**

1. Project Overview

CampusVolt is a full-stack web application for managing a small fleet of electric shuttle vehicles that move students around a campus. It brings four day-to-day concerns of a campus transport office into one browser-based dashboard: which vehicles exist and which route each one serves, which trips are scheduled or running and where the vehicles are, how full each vehicle is, and which problems need attention right now.

Problem addressed. Without a shared system, shuttle information is usually kept in spreadsheets or passed on by phone. Students cannot see where a shuttle is, administrators find out about overcrowding or late departures only after complaints, and route and vehicle details easily go out of date. CampusVolt keeps this information in one place and computes the important signals — progress, estimated arrival time, occupancy level and alerts — automatically.

Intended users. The code defines two roles. An **ADMIN** (transport staff) manages vehicles and routes, schedules and controls trips, and reports passenger counts. A **STUDENT** can view vehicles, routes, trips, live (simulated) positions, occupancy and alerts but cannot change anything. Anyone can register, but new accounts are always students; the first administrator is created at start-up from configuration.

Main objectives:

- Build a secure, role-based web application with a React frontend and Spring Boot REST APIs.
- Split the backend into independently runnable microservices, each with its own PostgreSQL database accessed through JPA/Hibernate.
- Implement realistic business rules: route distance estimation, trip life-cycle, simulated tracking with ETA, occupancy levels and self-resolving alerts.
- Return clear JSON error messages and validate every input on the server.

Completed application. The finished system consists of four Spring Boot services (authentication, fleet and routes, trips and tracking, occupancy and alerts) and a single-page React application with Login, Dashboard, Fleet, Trips, Occupancy and Profile pages. Vehicle movement is **simulated** on the server — no GPS hardware or external map service is used — and the application labels these positions as simulated in both the API and the UI.

2. Architecture and Microservices

CampusVolt follows a **microservice architecture**. Each backend service is a separate Maven project with its own `main` class, its own port and its own PostgreSQL database (created by `database/create-databases.sql`). A service never reads another service's database; when it needs other data it calls that service's REST API and forwards the caller's JWT. There is no API gateway: the React app reads the four service URLs from `VITE_*` environment variables and calls each service directly. All services are run locally (`mvn spring-boot:run`); the repository contains no deployment configuration.

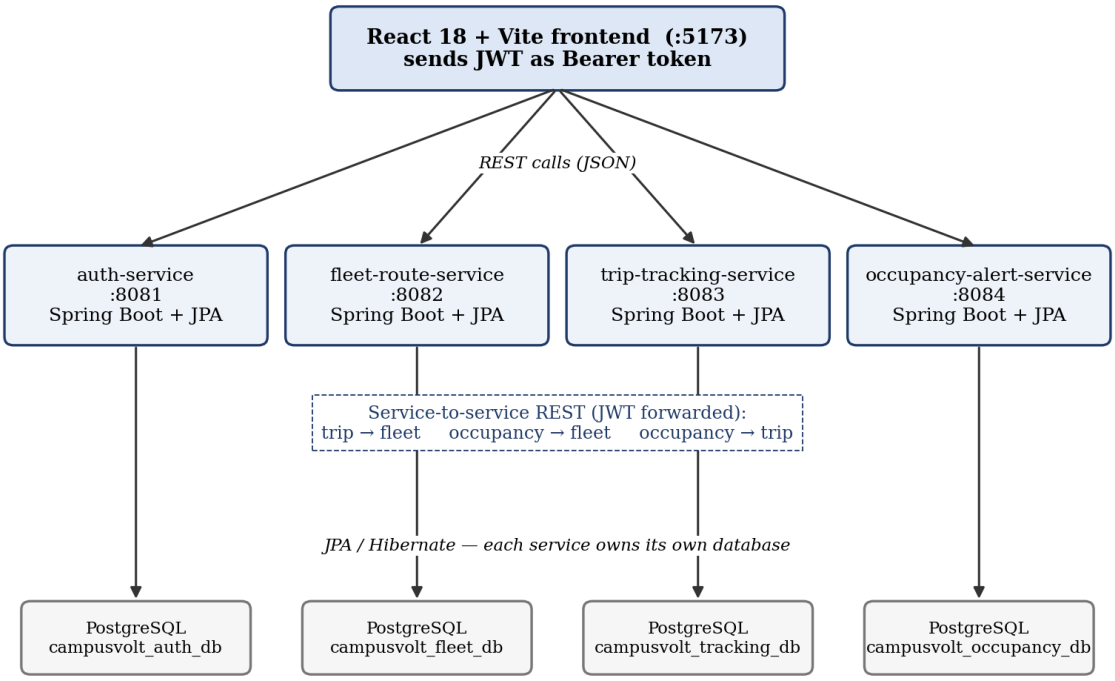


Figure 1. CampusVolt architecture — frontend, four services, their databases and service-to-service calls

2.1 Auth Service (port 8081)

Purpose: user registration, login and identity. **Data:** `users` table (full name, unique email, BCrypt password hash, role, created time). **Endpoints:** `POST /api/auth/register`, `POST /api/auth/login`, `GET /api/auth/me`. **Input/Output:** takes name, email and password; returns a signed JWT plus the user's profile. On start-up a `CommandLineRunner` creates the first ADMIN account from environment variables. The token it issues is what allows the user to call every other service.

2.2 Fleet and Route Service (port 8082)

Purpose: the master data of the fleet. **Data:** `vehicles`, `routes` and `route_stops`. **Endpoints:** `GET|POST /api/vehicles`, `GET|PUT|DELETE /api/vehicles/{id}`, and the same set for `/api/routes`. Write operations are ADMIN-only. It validates unique registration numbers and route names, keeps stops in order, and estimates route distance from stop coordinates when the admin does not enter one. On first start it seeds two sample routes and three sample vehicles. The other two services read vehicle and route details from this service.

2.3 Trip and Tracking Service (port 8083)

Purpose: scheduling and running trips. **Data:** `trips` (vehicle and route snapshot, status, times, progress, position, speed, pause flag). **Endpoints:** `GET|POST /api/trips`, `GET|PUT /api/trips/{id}`, `GET /api/tracking/vehicles`. When a trip is created it calls the fleet service to check the vehicle and copy the route's stops. A scheduled task moves running trips every 5 seconds, and each response includes progress, delay, staleness and ETA.

2.4 Occupancy and Alert Service (port 8084)

Purpose: passenger load and operational alerts. **Data:** `vehicle_occupancy` and `alerts`. **Endpoints:** `GET /api/occupancy`, `PUT /api/occupancy/{vehicleId}` (ADMIN), `GET`

`/api/alerts?includeResolved=`. It calls the fleet service for vehicle capacity and the trip service for trip status, then opens or resolves Overcrowding, Delay and Stale-tracking alerts.

2.5 React Frontend (port 5173)

A single-page application built with React 18, React Router and Vite. `services/api.js` holds every REST call, attaches the bearer token and turns error responses into readable messages; `AuthContext` keeps the logged-in user; `RouteMap.jsx` draws a route and the simulated vehicle position as an SVG. The UI hides admin-only buttons from students, but permission is always enforced again by the backend.

3. Technology Stack and Rationale

Technology	Where it is used in CampusVolt	Why it was chosen
Java 21	All four backend services; DTOs are written as Java `record`s, trip status rules use `switch` expressions	Strong typing, mature ecosystem, modern concise syntax
Spring Boot 3.5.6	Starts each service with an embedded server, reads `application.properties`, wires components	Removes boilerplate setup; each microservice runs with one command
Spring Web (REST)	`@RestController` classes, `RestClient` for service-to-service calls	Standard, annotation-based way to expose and consume JSON APIs
Spring Data JPA + Hibernate	Entities such as `Vehicle`, `Route`, `Trip`, `Alert`; repository interfaces	Maps Java objects to tables without hand-written SQL
PostgreSQL	Four databases, one per service	Reliable relational database with good Spring support
Spring Security + JJWT 0.12.6	BCrypt password hashing, JWT filter, `@PreAuthorize` role checks	Stateless authentication shared by independent services
Bean Validation	`@NotBlank`, `@Email`, `@Min` etc. on request records	Rejects bad input before business logic runs
React 18, React Router 6	Pages, components, navigation and session context	Component-based UI, fast updates without page reloads
Vite 5, HTML, CSS, JavaScript	Dev server and build; plain CSS in `index.css`	Fast development; no extra UI library needed
Maven, npm	`pom.xml` per service; `package.json` for the frontend	Standard dependency management and build tools

How Java, Spring Boot, JPA, Hibernate and the database fit together. Java is the language the services are written in. Spring Boot is the framework that turns those Java classes into a running web server: it receives HTTP requests, routes them to controller methods and converts Java objects to and from JSON. **JPA** (Jakarta Persistence) is the standard specification — the annotations `@Entity`, `@Id`, `@OneToMany`, `@ManyToOne`, `@Enumerated` describe how a class maps to a table. **Hibernate** is the implementation of JPA that Spring Boot uses: it generates the SQL, tracks changes to loaded objects and writes them back when a transaction commits. With `ddl-auto=update`, Hibernate also creates the tables on first start. **Spring Data JPA** sits on top, so an interface method such as `existsByRegistrationNumberIgnoreCaseAndIdNot(...)` is turned into a query automatically. The PostgreSQL JDBC driver finally carries those SQL statements to the database.

4. Detailed Feature Implementation

Features are grouped by the service that implements them.

4.1 Registration, Login and Role-Based Access (Auth Service + all services)

Purpose: Lets users create an account and sign in, and makes sure students can only view data while admins can change it.

Implementation: `AuthController` exposes register, login and me. `AuthService` checks for a duplicate email (409), hashes the password with BCrypt and saves a `User` with role STUDENT. On login it compares the password hash and asks `JwtService` to sign a token containing email, name and role. Every service has its own `JwtAuthFilter` that verifies the signature with the shared `JWT_SECRET` and places the user and role into the Spring Security context; controller methods marked `@PreAuthorize("hasRole('ADMIN')")` then reject students with 403. On the frontend, `LoginPage` and `AuthContext` store the token and `api.js` adds it as `Authorization: Bearer ...` to every request.

Java Technology Used: Spring Security (filter chain, `PasswordEncoder`, method security), JJWT for HMAC-SHA256 signing, JPA entity `User` with a unique email column, Bean Validation on `RegisterRequest`/`LoginRequest`, and an `ApiExceptionHandler` that returns `{"message": ...}`.

Working Flow: User submits the form → `POST /api/auth/login` → validation → user lookup by email (`UserRepository`) → BCrypt check → JWT returned → frontend saves token → later calls to any service carry the token → that service's filter verifies it locally.

Design Rationale: Because the token is self-contained and signed, each microservice can verify users **without calling the auth service**, which keeps services independent. Forcing self-registered accounts to STUDENT prevents anyone from making themselves an admin.

Result: Secure sign-in with hashed passwords, a role-aware UI, and server-enforced permissions on every write endpoint.

4.2 Vehicle and Route Management (Fleet and Route Service)

Purpose: Gives administrators one place to maintain the fleet: vehicles (registration, name, capacity, status, battery level) and routes with ordered stops and coordinates.

Implementation: `VehicleController` / `RouteController` delegate to `VehicleService` / `RouteService`. Entities: `Vehicle` (`@ManyToOne` to `Route`), `Route` (`@OneToMany(mappedBy = "route", cascade = ALL, orphanRemoval = true)` with `@OrderBy("sequence ASC")`) and `RouteStop`. On save, stops are rebuilt from the request in order; registration numbers are upper-cased and checked for uniqueness; deleting a route first unassigns its vehicles. `FleetPage.jsx` provides the tables and add/edit/delete forms.

Java Technology Used: JPA relationships and cascades, `@Enumerated(EnumType.STRING)` for `VehicleStatus` (ACTIVE, CHARGING, MAINTENANCE, OUT_OF_SERVICE), derived query methods, `@Transactional` services, `ResponseStatusException` for 404/409 errors.

Working Flow: Admin fills the vehicle form → `POST /api/vehicles` → validation → duplicate check → route lookup → `save()` → Hibernate INSERT → JSON `VehicleResponse` → table refreshes.

Design Rationale: Cascading stops from the route means a route and its stops are always saved or deleted together, so no orphan stops can exist. Returning DTOs instead of entities keeps the API stable and avoids exposing lazy-loaded relations.

Result: A complete CRUD module for vehicles and routes with clear validation messages, available read-only to students.

4.3 Automatic Route Distance Estimation (Fleet and Route Service)

Purpose: Tracking and ETA need a route length, but admins may not know the exact distance.

Implementation: In `RouteService.toResponse`, if `distanceKm` is empty and the route has at least two stops, the service adds up the **haversine** (great-circle) distance between consecutive stops and returns it with `distanceEstimated = true`. The UI labels such values as "estimated".

Java Technology Used: Plain Java maths (`Math.sin`, `Math.asin`, Earth radius 6371 km) inside a Spring service; the stops come from the JPA `@OneToMany` collection already ordered by Hibernate.

Working Flow: Route requested → stops loaded in sequence → distance computed if missing → returned with an estimated flag.

Design Rationale: Computing the value at read time (rather than storing it) means the estimate always matches the current stops.

Result: Every route with two or more stops has a usable distance, honestly marked as entered or estimated.

4.4 Trip Scheduling and Life-Cycle (Trip and Tracking Service)

Purpose: Lets admins schedule trips and move them through a controlled sequence of states.

Implementation: `TripService.create` calls `FleetClient` (a Spring `RestClient`) to fetch the vehicle and route from the fleet service, rejects vehicles in MAINTENANCE or OUT_OF_SERVICE, and stores a snapshot of vehicle name, route name, distance and the stop path in the `Trip` entity. `changeStatus` uses a `switch` to allow only Scheduled → In progress → Completed and Scheduled/In progress → Cancelled, and returns 409 if the vehicle already has a running trip. Only scheduled trips can be rescheduled. `TripsPage.jsx` provides the schedule form and Start / Complete / Cancel / Pause buttons.

Java Technology Used: Service-to-service REST with JWT forwarding, JPA entity with `TripStatus` enum, repository query `existsByVehicleIdAndStatusAndIdNot`, transactional updates (Hibernate dirty checking writes the changed fields automatically).

Working Flow: Admin picks vehicle, route and time → `POST /api/trips` → trip service calls `GET /api/vehicles/{id}` and `GET /api/routes/{id}` on the fleet service → trip saved as SCHEDULED → later `PUT /api/trips/{id}` with a new status.

Design Rationale: Copying the route path into the trip keeps the trip service working on its own data and means editing a route later does not change trips already scheduled. If the fleet service is down, the user receives a clear 503 message.

Result: Reliable trip scheduling with enforced state rules and no double-booking of a vehicle.

4.5 Simulated Live Tracking, Delay and ETA (Trip and Tracking Service)

Purpose: Shows where each running shuttle is, how far along it is and when it should arrive.

Implementation: `TrackingSimulator` has a `@Scheduled(fixedRate = 5000)` method. Every 5 seconds it loads IN_PROGRESS trips, picks a random speed of 18–32 km/h, advances `progressKm`, interpolates a latitude/longitude along the stored path and completes the trip at the end. `TripService.toResponse` computes progress %, delay (more than 5 minutes past scheduled start), staleness (no update for over 2 minutes) and ETA = remaining distance ÷ speed, with an `etaNote` explaining when ETA cannot be shown (not running, unknown distance, paused, stale). `RouteMap.jsx` draws the route and vehicle as SVG and the Trips and Dashboard pages refresh periodically.

Java Technology Used: Spring scheduling (`@EnableScheduling`, `@Scheduled`), `@Transactional` so Hibernate saves every moved trip, `java.time` (`Instant`, `Duration`) for delay and staleness.

Working Flow: Admin starts trip → vehicle placed at first stop → scheduler moves it every 5 s → frontend polls `GET /api/trips` or `/api/tracking/vehicles` → map and "~N min" ETA update.

Design Rationale: A server-side simulation allows tracking to be demonstrated without GPS devices. Responses carry `simulated: true` so the data is never presented as real GPS.

Result: A live-feeling tracking view with progress, speed, delay and ETA that behaves sensibly in edge cases.

4.6 Passenger Occupancy Reporting (Occupancy and Alert Service)

Purpose: Shows how full each vehicle is so that overcrowding can be noticed and managed.

Implementation: `PUT /api/occupancy/{vehicleId}` (ADMIN) calls the fleet service through `ServiceClients` to get the vehicle's name and capacity, rejects counts above twice the capacity (and negative counts through validation), and saves a `VehicleOccupancy` row keyed by vehicle id with who updated it and when. The response gives the percentage and a level: NORMAL, NEAR_CAPACITY (85% or more) or OVERCROWDED (above capacity). `OccupancyPage.jsx` shows the levels and an update form.

Java Technology Used: JPA entity with an assigned `@Id` (the vehicle id) so each vehicle has exactly one occupancy row; `RestClient` with error mapping (404 vehicle not found, 503 fleet unavailable).

Working Flow: Admin enters count → PUT request → fleet lookup → plausibility check → save → overcrowding check → level returned.

Design Rationale: Taking capacity from the fleet service at update time keeps one source of truth for vehicle data.

Result: An up-to-date occupancy view for every reported vehicle with colour-coded levels.

4.7 Automatic Self-Resolving Alerts (Occupancy and Alert Service)

Purpose: Draws attention to problems — overcrowding, late trips and lost tracking — without manual monitoring.

Implementation: `AlertService` gives each condition a unique `conditionKey` (for example `DELAY:trip:7`). `raise()` updates the existing active alert for that key or creates a new one, so duplicates are avoided. Overcrowding is checked on every occupancy update. When alerts are listed, the service fetches trips from the trip service and opens Delay alerts (warning, critical from 15 minutes) and Stale-tracking alerts; alerts whose condition has cleared are marked resolved with a timestamp. If the trip service is unreachable the list still loads and the response says so.

Java Technology Used: JPA entity `Alert` with enums for type and severity and a database index on `(conditionKey, active)`; Spring Data methods such as `findByConditionKeyAndActiveTrue`, `findTop50ByOrderByCreatedAtDesc`.

Working Flow: Dashboard/Occupancy page requests `GET /api/alerts` → trips fetched → alerts opened/updated/resolved → active (or recent resolved) alerts returned → shown with severity.

Design Rationale: Keying alerts by condition makes the logic idempotent: repeating the check never creates duplicate alerts, and alerts close automatically when the problem disappears.

Result: An always-current alert list that combines data from three services.

4.8 Validation and Error Handling (all services)

Request DTOs are Java records annotated with Bean Validation constraints and checked with `@Valid`. Business errors are raised as `ResponseStatusException`. A per-service `@RestControllerAdvice` (`ApiExceptionHandler`) converts all of these into one consistent JSON

shape, `{"message": "..."}`, with status 400 (validation), 401 (missing/invalid token), 403 (wrong role), 404 (not found), 409 (conflict) or 503 (dependent service down). The frontend's `ApiError` class reads this message and shows it to the user, so errors are understandable rather than raw stack traces.

5. Individual Team Contributions

Source of this allocation. The project folder is not a Git repository, so there is no commit history to confirm authorship. The ownership below follows the "Team and ownership" table recorded in the project README, which assigns one backend service to each member and records the Auth Service and the React frontend as shared parts coordinated by Kawin A N. Where a part is shared, this is stated rather than credited to one person alone.

Member	Owned service (README)	Shared / integration responsibility
Kawin A N (2024115018)	Fleet and Route Service (8082)	Coordinated Auth Service and React frontend
Shankararam S U (2024115020)	Trip and Tracking Service (8083)	Fleet API integration; Trips page data and tracking view
Rohan Beniel E (2024115124)	Occupancy and Alert Service (8084)	Fleet + Trip API integration; Occupancy page and alerts data

5.1 Kawin A N — 2024115018

- **Main responsibility:** the fleet master data that every other service depends on, plus coordination of the shared authentication service and the React frontend.
- **Features:** vehicle CRUD, route CRUD with ordered stops, haversine distance estimation, sample-data seeding; registration, login, JWT issuing and the first-admin seeding.
- **Frontend:** application shell (`Layout`, routing in `App.jsx`), `AuthContext`, `api.js`, Login, Profile and Fleet pages, shared UI components and styles.
- **Spring Boot:** `VehicleController`, `RouteController`, their services, `AuthController`/`AuthService`, the JWT filter and security configuration pattern reused by all services.
- **Hibernate/JPA:** `Vehicle`–`Route` many-to-one and `Route`–`RouteStop` one-to-many mappings with cascade and ordering; `User` entity with unique email; derived queries for uniqueness checks.
- **Integration and testing:** defined the vehicle/route JSON that the trip and occupancy services consume; verified 401/403 behaviour across services. No automated test classes exist in the repository; checks were done by running the application.
- **Outcome:** a secure entry point and a reliable source of vehicle and route data for the whole system. **Technologies:** Java 21, Spring Boot, Spring Security, JJWT, JPA/Hibernate, PostgreSQL, React, Vite.

5.2 Shankararam S U — 2024115020

- **Main responsibility:** the trip and tracking service — everything about trips once vehicles and routes exist.
- **Features:** trip scheduling, status life-cycle rules, one-running-trip-per-vehicle rule, rescheduling, pause/resume, simulated movement every 5 seconds, progress, delay, stale detection and ETA with explanatory notes.

- **Frontend:** the Trips page is the user-facing view of this service; it consumes the trip and tracking endpoints (schedule form, action buttons, `RouteMap` position display). The README records the frontend as shared, so this work was integrated into the common React app.
- **Spring Boot:** `TripController`, `TripService`, `TrackingSimulator` (`@Scheduled`), and `FleetClient` using `RestClient` with JWT forwarding and 400/503 error mapping.
- **Hibernate/JPA:** the `Trip` entity storing a snapshot of route data and the path as text, `TripStatus` enum, repository queries such as `findByStatus` and `existsByVehicleIdAndStatusAndIdNot`; transactional updates for each simulation tick.
- **Integration and testing:** depends on the fleet service API and supplies trip data (delay, staleness) to the occupancy service's alert logic, so its response format must stay compatible with that service.
- **Outcome:** the live, moving part of the application that users watch. **Technologies:** Java 21, Spring Boot (Web, Scheduling), JPA/Hibernate, PostgreSQL, RestClient, React.

5.3 Rohan Beniel E — 2024115124

- **Main responsibility:** the occupancy and alert service, which turns data from the other services into operational warnings.
- **Features:** passenger count reporting with plausibility checks, occupancy percentage and levels, Overcrowding, Delay (warning/critical) and Stale-tracking alerts that open and resolve automatically, resolved-alert history.
- **Frontend:** the Occupancy page and the alert information shown on the Dashboard consume this service's endpoints (update form, level badges, active/resolved alert list), integrated into the shared React app.
- **Spring Boot:** `OccupancyController`, `OccupancyService`, `AlertService`, and `ServiceClients`, which calls both the fleet and trip services and degrades gracefully if the trip service is unavailable.
- **Hibernate/JPA:** `VehicleOccupancy` with the vehicle id as an assigned primary key, `Alert` with type/severity enums, a composite index on `conditionKey` and `active`, and queries for active, typed and latest alerts.
- **Integration and testing:** the most cross-service module — it consumes two other APIs, so it depends on JWT forwarding and error mapping working correctly between services.
- **Outcome:** the monitoring layer that tells administrators what needs attention. **Technologies:** Java 21, Spring Boot, JPA/Hibernate, PostgreSQL, RestClient, Bean Validation, React.

How the parts fit together. Kawin's fleet service is the base data source and the auth service makes every request trusted; Shankararam's trip service builds on fleet data to produce movement and timing; Rohan's occupancy service combines fleet and trip data into occupancy levels and alerts. Each member can therefore explain a complete vertical slice — page, endpoints, business rules and database mapping — of their own service.

6. End-to-End Application Workflow

A typical session from a user's point of view:

1. **Sign in.** The user opens the React app and logs in. The auth service verifies the password and returns a JWT, which the frontend stores and attaches to every later request.
2. **Prepare the fleet.** The admin opens Fleet, adds a route with stops and a vehicle assigned to it. The fleet service's controller validates the request, the service applies the rules, and Hibernate inserts rows into `routes`, `route_stops` and `vehicles`.

3. **Schedule and start a trip.** On Trips, the admin schedules a trip; the trip service fetches vehicle and route from the fleet service and saves the trip. Pressing Start sets it IN_PROGRESS and places the vehicle at the first stop.
4. **Watch it move.** Every 5 seconds the scheduler advances the trip. The Trips and Dashboard pages poll the API and redraw the map, progress and ETA.
5. **Report passengers.** On Occupancy, the admin enters a passenger count. The occupancy service checks capacity with the fleet service, saves the value and opens an Overcrowding alert if needed.
6. **See alerts.** The dashboard asks for alerts; the occupancy service checks trips for delays or stale tracking, updates the `alerts` table and returns the active list. A student sees the same information but without edit buttons.

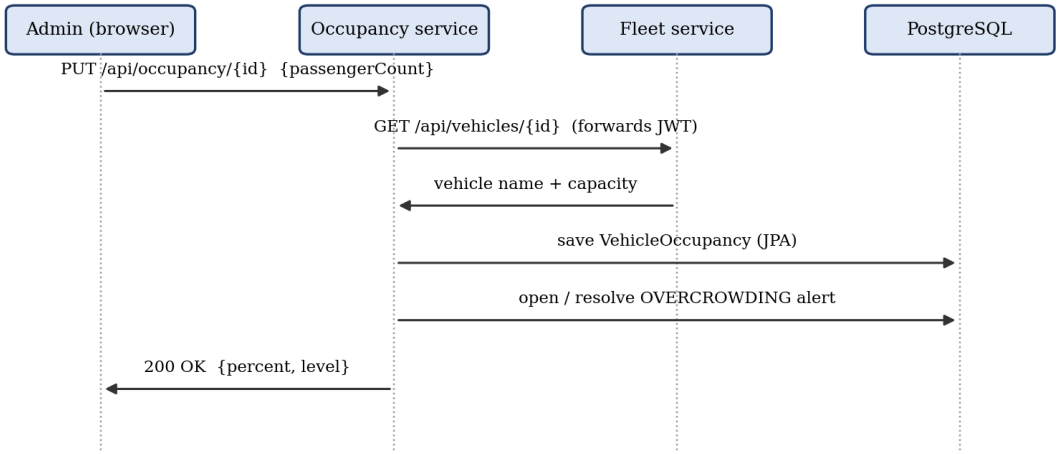


Figure 2. Sequence for reporting a passenger count (example of a cross-service request)

7. Final Deliverables and Implemented Features

Implemented

- **Frontend:** React single-page app with Login/Register, Dashboard (summary and alerts with auto-refresh), Fleet (vehicles and routes CRUD), Trips (schedule, control, SVG route map with simulated position), Occupancy and Profile pages; role-aware buttons.
- **Backend:** four independently runnable Spring Boot services exposing the REST endpoints listed in Section 2.
- **Persistence:** four PostgreSQL databases; seven JPA entities and five enums mapped by Hibernate, including one-to-many and many-to-one relationships, enum columns and an index; tables generated automatically.
- **Integration:** service-to-service REST calls with JWT forwarding and clear 503 handling when a dependency is down.
- **Security and validation:** BCrypt passwords, signed JWTs verified in every service, ADMIN/STUDENT rules with `@PreAuthorize`, Bean Validation, and uniform JSON error messages.
- **Business logic:** distance estimation, trip state machine, single running trip per vehicle, simulated tracking, ETA, occupancy levels and self-resolving alerts.

Limitations and possible enhancements

- Vehicle positions are **simulated**; no GPS device or map tile service is connected.

- There is no API gateway or service discovery; service URLs are configured through environment variables.
- The repository contains **no automated tests** and no deployment configuration; the system has been run locally only.
- A development default is used for the JWT secret if none is set, so a strong secret must be configured outside local development.
- Possible next steps: real GPS integration, an API gateway, automated unit/integration tests and containerised deployment.

Conclusion

The team built CampusVolt, a working campus EV fleet system made of a React frontend and four Spring Boot microservices, each with its own PostgreSQL database managed through JPA and Hibernate. The project applies the core ideas of the Web Programming course — client–server communication over HTTP, REST API design, a component-based frontend, server-side validation, authentication with tokens, role-based authorisation and object-relational persistence — and extends them to communication between independent services. Each member owns one service from the user interface down to the database, and together the three services form one complete application.